



INDIA.RUNS

Build what next India runs on

CV_ana – Resume Understanding & Candidate Ranking

Team: Cosmic_Code

Team Leader: Indraneel Mandal

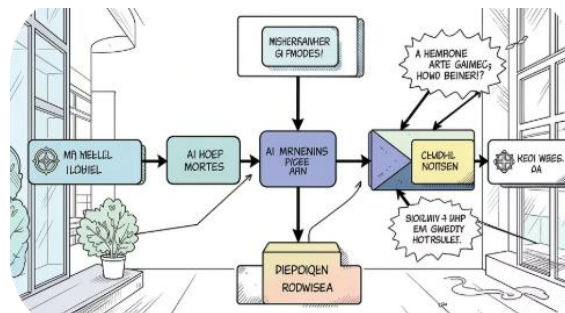
Problem statement: Build an automated system that ingests job descriptions (JDs) and candidate CVs, extracts structured signals, and returns an explainable ranked shortlist optimized for recruiter relevance and runtime constraints.

Solution Overview



What we built

A pipeline that parses Job Descriptions and CVs, extracts structured skill/experience entities, computes multi-signal relevance scores, and returns a ranked, auditable candidate list through a lightweight API and UI.



How it's different

Combines semantic embeddings, rule-based extraction, and feature-level heuristics to go beyond keyword matching — adds contextual fit, seniority normalization, and explainable evidence for each ranking decision.

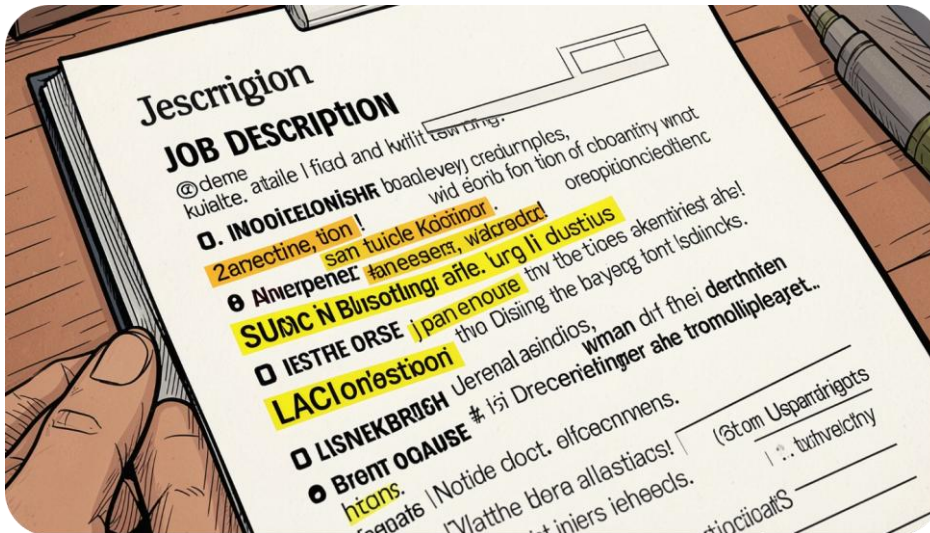
JD Understanding & Candidate Evaluation

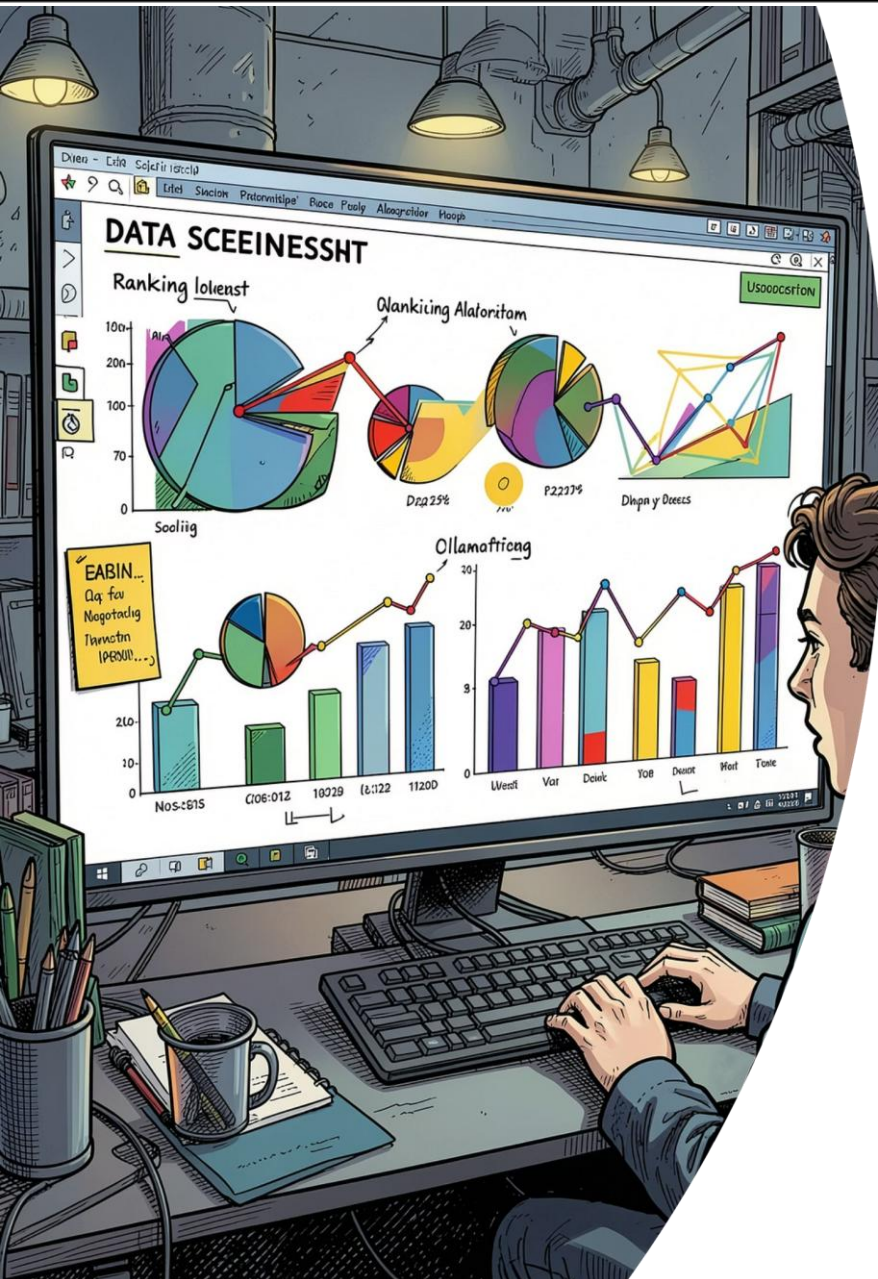
Key JD requirements (extracted)

- Core skills: required technologies, domain keywords, tooling
- Experience: years, seniority level, leadership vs IC
- Role signals: deliverables, responsibilities, remote/location constraints
- Soft constraints: communication, collaboration, certifications

Candidate signals evaluated

- Semantic skill match (embeddings + taxonomy)
- Experience depth & recency (normalized years & project context)
- Role alignment: titles → responsibilities mapping
- Quality signals: education, open-source, portfolio, verification flags
- Behavioral hints from text: leadership, ownership, domain-specific verbs





Ranking Methodology

1. Retrieval

Fast candidate retrieval using inverted skill index + approximate nearest neighbors (ANN) over semantic embeddings to surface a candidate set (~K candidates).

2. Feature extraction

Per-candidate signals: embedding similarity, skill coverage, seniority match, role relevance, recency, quality scores (portfolio, publications), and fidelity flags.

3. Scoring & aggregation

Weighted linear ensemble (configurable) combining ML scorer (pairwise ranker) and deterministic heuristics; optional learning-to-rank model for production tuning.

4. Rerank & business rules

Hard constraints (location, visa) and freshness boosts applied last, producing final ranked list with per-signal provenance.

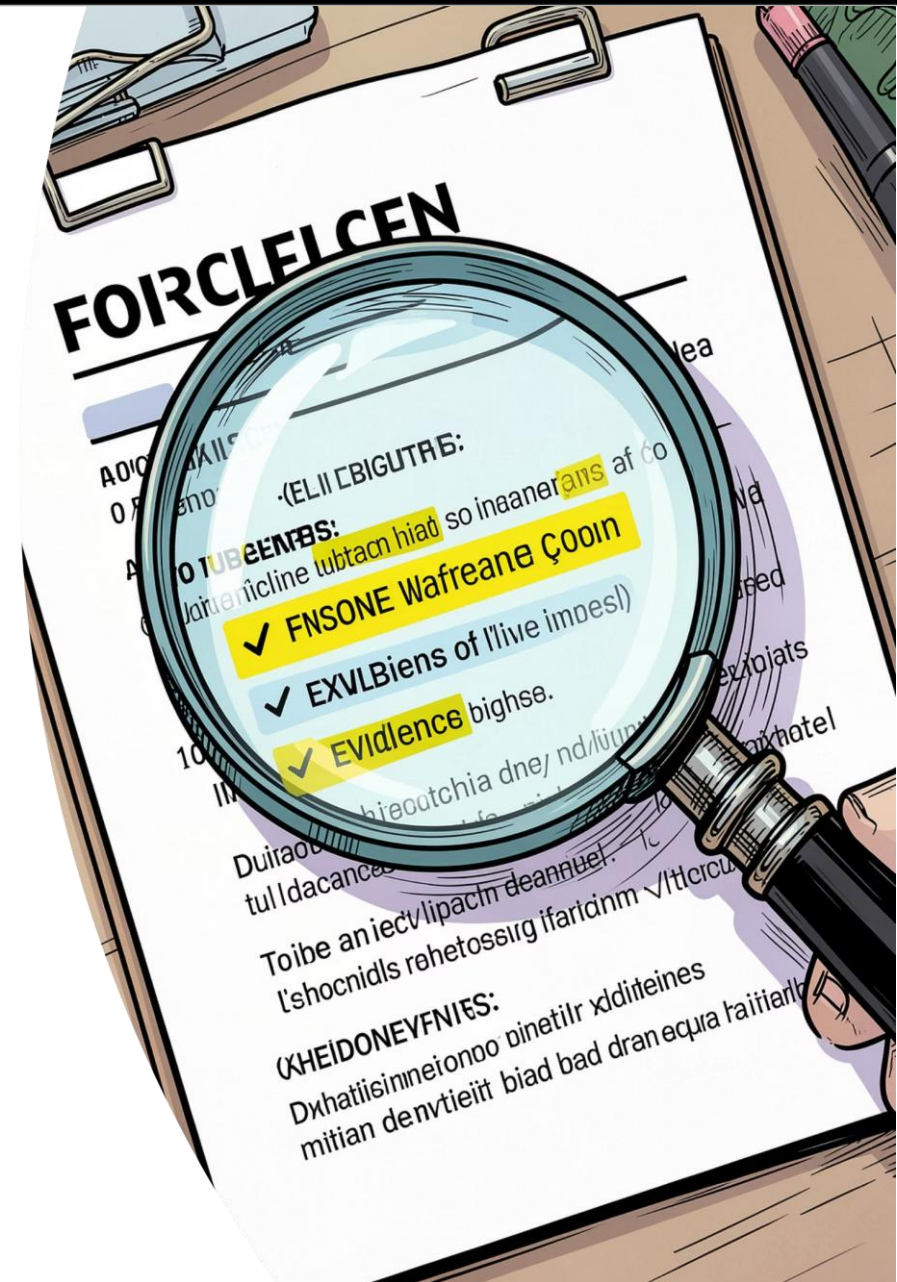
Explainability & Data Validation

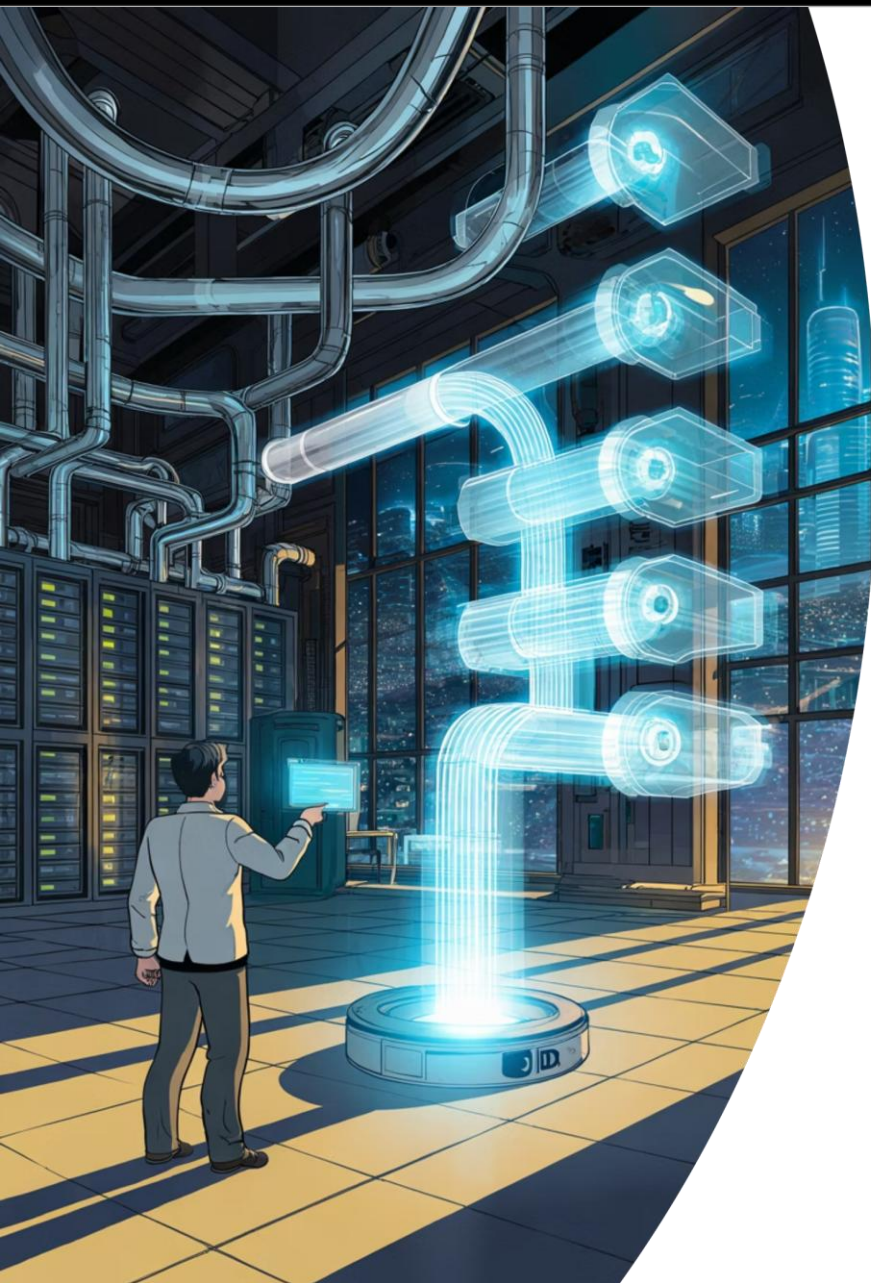
Explainability

- Per-candidate evidence: top matching sentences, matched skill phrases, and contribution to final score.
- Transparent weights: show normalized contribution of embedding vs rule-based features.
- Human-readable justification templates generated from evidence (no hallucinated claims).

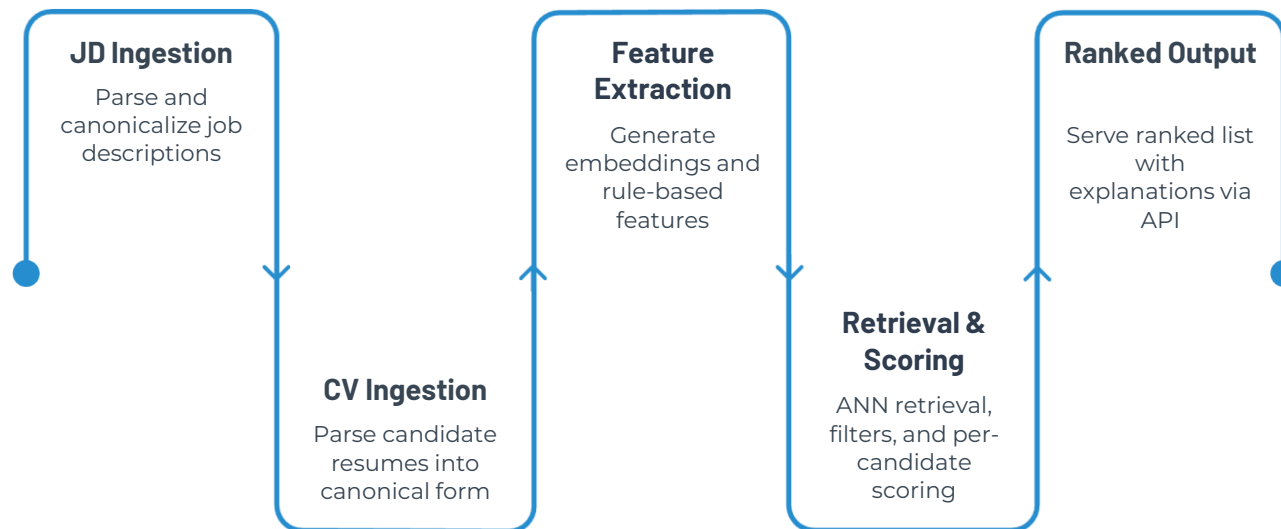
Data validation & safety

- Schema checks and heuristics to detect low-quality/duplicate/resume-parsing errors.
- Field-level confidence scores: flag suspicious dates, inflated titles, or empty critical fields.
- Conservative fallback: when evidence is weak, surface "needs verification" and deprioritize in rank.



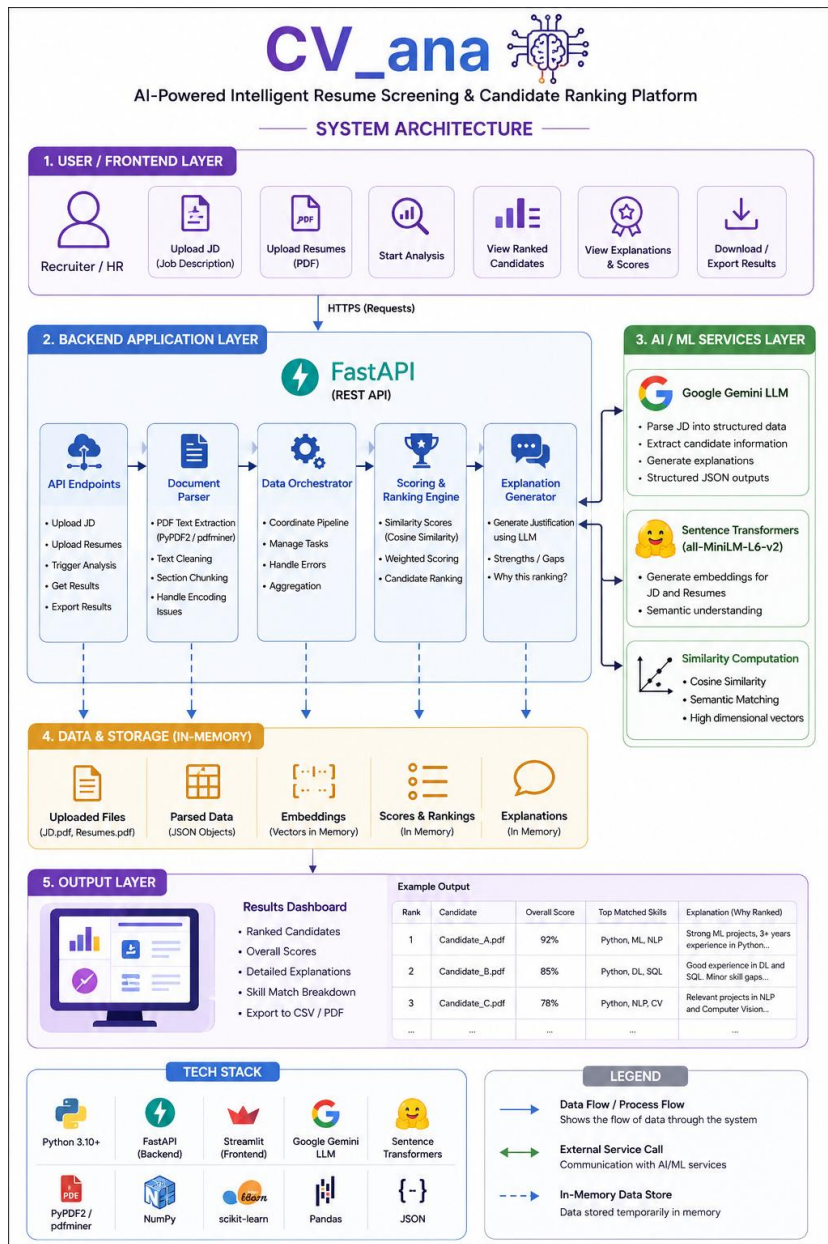


End-to-End Workflow



The system runs asynchronously:

JD arrives → canonical JD object → retrieval (ANN + filters) → per-candidate scoring → explainable ranked results served via REST API and lightweight UI.



System Architecture

Core components:

- Frontend: simple React UI for JD upload and shortlist inspection
- API Gateway: routes requests, enforces quotas
- Parsing Service: resume & JD parsers (rule-based + regex + NLP)
- Feature Service: embedding model, skill normalization, signal calculator
- Ranking Service: ensemble scorer and LTR model
- Persistence: vector DB for embeddings, relational DB for metadata and audit logs

Deployment notes:

- Containerized microservices (Docker) + Kubernetes for horizontal scaling
- Lightweight GPU usage limited to embedding inference for cost control
- Observability: metrics (latency, throughput), audit logs for explainability

Results & Performance

Example evaluation strategy and metrics to demonstrate ranking quality:

Precision@K

Measure recruiter accept rate on top-K suggestions using human-labeled relevance.

NDCG

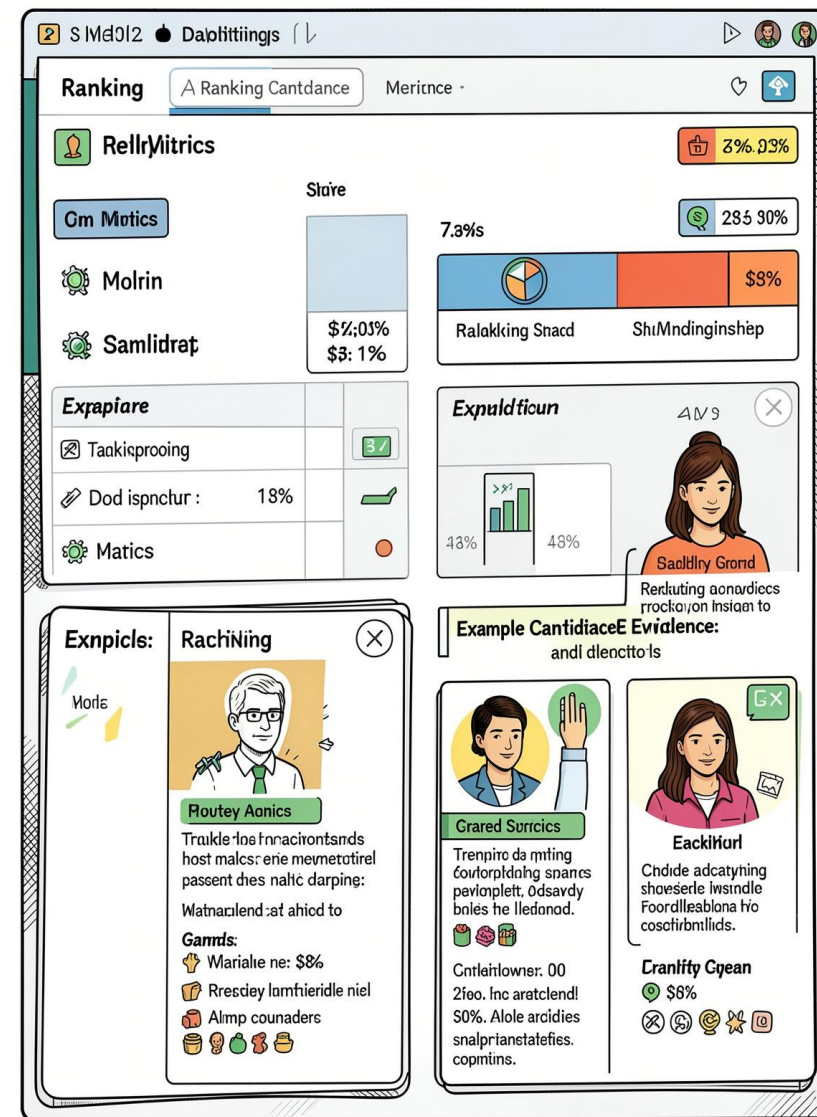
Rank-aware metric for graded relevance across candidates.

Latency & Cost

Target end-to-end runtime under challenge constraints (e.g., <200ms retrieval, configurable scoring budget).

Robustness

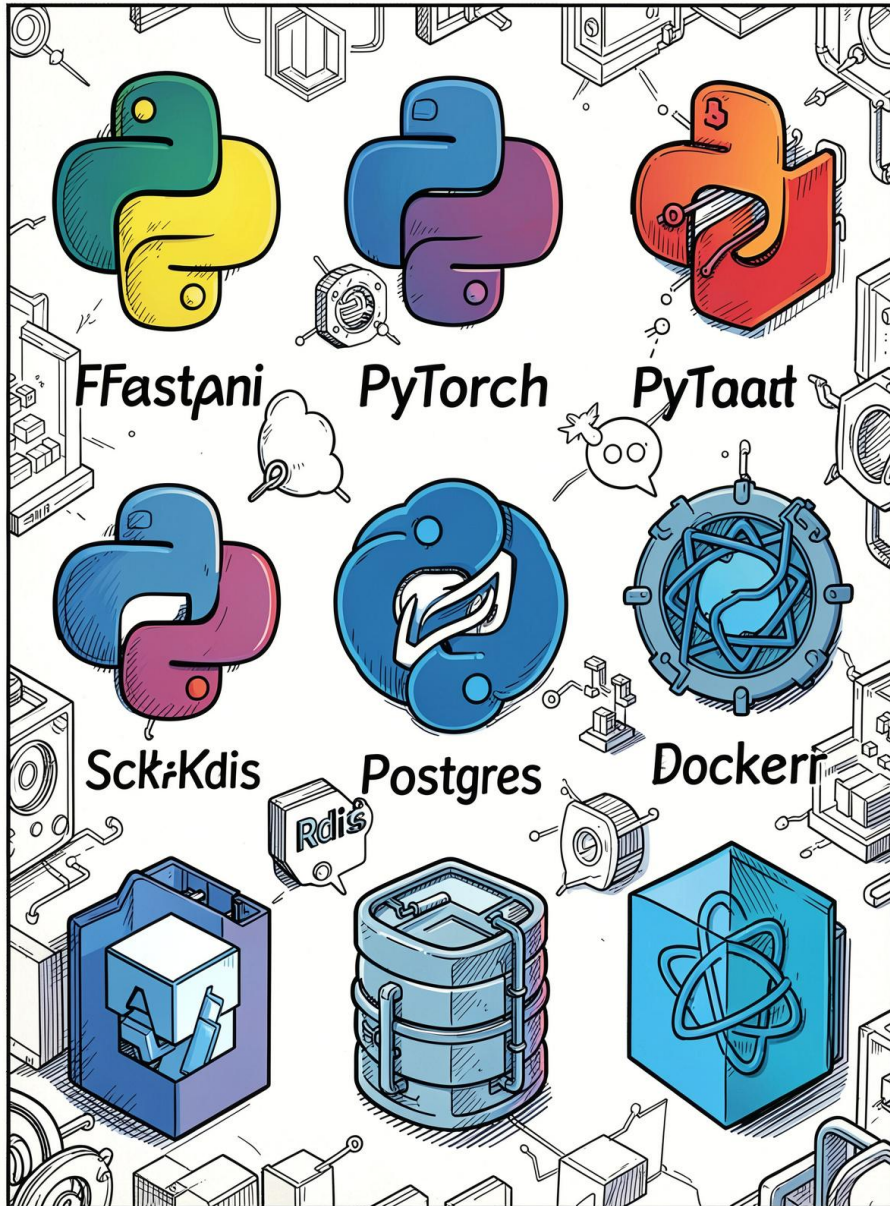
Stress tests with incomplete/ambiguous CVs; conservative fallback policies preserve precision.



Technologies & Why They Were Chosen

- **Python** — primary language for NLP, parsers, and model orchestration
- **FastAPI** — lightweight, async REST API for low-latency serving
- **PyTorch / Transformers** — embedding and ranking model building and inference
- **Vector DB (FAISS/Weaviate)** — ANN retrieval for semantic search
- **Postgres** — metadata, audit logs, and candidate profiles
- **Redis** — caching hot results and short-lived state
- **Docker & Kubernetes** — reproducible deployment and scaling
- **React** — recruiter UI for inspection, filtering, and feedback

Choices prioritize reproducibility, low-latency retrieval, explainability, and incremental deployability for contest constraints.



Submission Assets

GitHub: https://github.com/OrbIndraneel/CV_ana

 redrob |  H2S

INDIA.RUNS

Build what next India runs on

THANK YOU